

UNIT - 7 : XML & JSON

Introduction and use of XML

1. INTRODUCTION TO XML

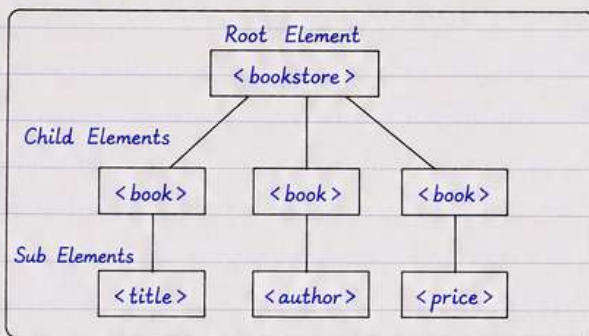
- XML (eXtensible Markup Language) एक Markup Language है जिसका उपयोग Data को Store करने और Transport करने के लिए किया जाता है।
- XML को W3C (World Wide Web Consortium) द्वारा Recommend किया गया है।
- XML का Design ऐसे किया गया है कि यह Platform Independent हो, यानी यह किसी भी Operating System और Application पर काम करता है।
- XML का उद्देश्य Data को Readable, Shareable और Self-descriptive बनाना है।

XML की विशेषताएँ

- Extensible
- Platform Independent
- Text Based
- Hierarchical Structure
- Easy to Parse
- Language Independent

2. XML की बेसिक संरचना

XML Document में एक Root Element होता है और उसके अंदर कई Child Elements हो सकते हैं।



XML Syntax Rules

- XML Case Sensitive होता है।
- XML Document में एक ही Root Element होता है।
- हर Element का Properly Open और Close Tag होना चाहिए।
- हर Tag Proper Nesting में होना चाहिए।
- Attribute का मान हमेशा Quotes में होता है।
- Special Characters जैसे <, >, &, ' , ' ' का Use Valid Form में करना होता है।

3. XML DOCUMENT EXAMPLE

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book id="101">
    <title> XML Introduction </title>
    <author> BK LearnX </author>
    <price> 450 </price>
  </book>
  <book id="102">
    <title> Web Technology </title>
    <author> John Doe </author>
    <price> 350 </price>
  </book>
</bookstore>
```

- <?xml version="1.0" encoding="UTF-8"?> → XML Declaration
- <bookstore> → Root Element
- <book> → Child Element
- <title>, <author>, <price> → Sub Elements
- id → Attribute of <book> Element

4. XML के USES (उपयोग)

- Data Storage: XML का उपयोग Data को Structured Format में Store करने के लिए होता है।
- Data Transport: XML का उपयोग Internet के माध्यम से Data को भेजने और प्राप्त करने के लिए किया जाता है।
- Configuration Files: Software और Application की Settings को Store करने के लिए XML Files का उपयोग होता है।
- Web Services: XML का उपयोग Web Services (SOAP, REST) में Request और Response Message Format के लिए होता है।
- Document Sharing: XML अलग-अलग Platform के बीच Document और Data Share करने में मदद करता है।

5. XML के BENEFITS

Extensible	Self-Descriptive	Platform Independent	Data Integration	Easy Maintenance
User अपने अनुसार Tags बनाकर Data Store कर सकते हैं।	XML Document खुद ही अपनी Structure और Data को Describe करता है।	XML किसी भी OS, Hardware या Software पर काम करता है।	XML अलग-अलग Systems और Applications के बीच Data Integration आसान बनाता है।	XML Files को पढ़ना, समझना और Modify करना आसान होता है।

Summary : XML एक Powerful और Flexible Markup Language है जिसका उपयोग Data को Store, Transport और Share करने के लिए किया जाता है। यह Web और Enterprise Applications में बहुत महत्वपूर्ण भूमिका निभाता है।

DIFFERENCE BETWEEN XML AND HTML

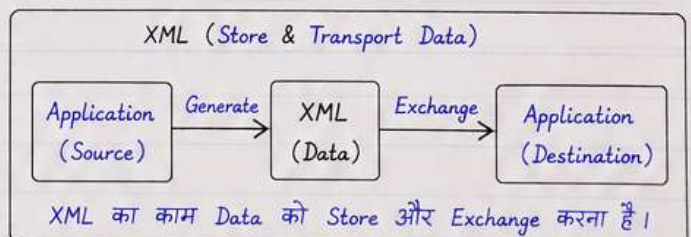
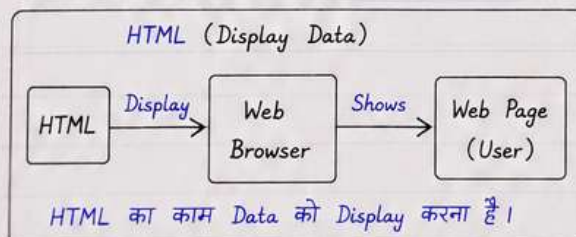
1. INTRODUCTION

- XML (eXtensible Markup Language) और HTML (HyperText Markup Language) दोनों Markup Language हैं, लेकिन इनका उद्देश्य और उपयोग अलग-अलग होता है।
- HTML का उपयोग Web Page की Structure और Display के लिए किया जाता है।
- XML का उपयोग Data को Store, Transport और Share करने के लिए किया जाता है।
- नीचे दोनों के बीच मुख्य अंतर दिए गए हैं।

2. DIFFERENCE BETWEEN XML AND HTML

Basis	XML	HTML
1. Full Form	eXtensible Markup Language	HyperText Markup Language
2. Purpose (उद्देश्य)	Data को Store, Transport और Share करना।	Web Page की Structure और Display करना।
3. Design by	XML को W3C द्वारा Design किया गया है।	HTML को W3C द्वारा Design किया गया है।
4. Nature	XML Platform Independent होता है, किसी भी OS पर काम करता है।	HTML Platform Dependent होता है, किसी Specific Browser पर काम करता है।
5. Tags	XML में User अपने खुद के Tags Define कर सकता है।	HTML में Predefined Tags होते हैं, User नए Tags Define नहीं कर सकता।
6. Syntax Rules	XML Strict Rules Follow करता है, हर Tag को Properly Close करना ज़रूरी होता है।	HTML Forgiving होता है, कुछ गलतियों होने पर भी Browser उसे Ignore कर देता है।
7. Error Handling	XML Errors को Display करता है, क्योंकि यह Strict होता है।	HTML Errors को Ignore कर देता है और Page को फिर भी Display कर देता है।
8. Data Focus	XML Data पर Focus करता है।	HTML Data की Presentation पर Focus करता है।
9. Readability	XML Plain Text Format में होता है, Read और Write करना आसान है।	HTML भी Readable होता है लेकिन यह Mostly Display के लिए होता है।
10. Use	XML का उपयोग Web Services, Configuration Files, Data Exchange में किया जाता है।	HTML का उपयोग Web Pages, Websites और Web Applications के Front-end में किया जाता है।
11. Example	<pre><book> <title> BK LearnX </title> <author> Brajkant </author> </book></pre>	<pre><html> <body> <h1> BK LearnX </h1> <p> Welcome to BK LearnX </p> </body> </html></pre>

3. VISUAL COMPARISON



4. KEY POINTS

- HTML Web Page को बनाने के लिए Use होता है।
- XML Data को Structured Format में Store और Share करने के लिए Use होता है।
- HTML Presentation Oriented है, जबकि XML Data Oriented है।
- XML को किसी भी Programming Language द्वारा Read और Process किया जा सकता है।

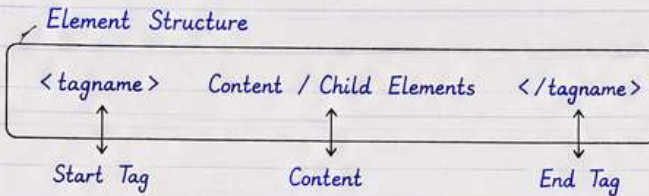
Remember

HTML = Page Display
XML = Data Exchange

XML ELEMENTS, ATTRIBUTE, NAME SPACE

1. XML ELEMENTS (तत्व)

- XML Document की Building Block Elements होते हैं।
- हर Element एक Start Tag और End Tag के बीच define किया जाता है।
- Element के अंदर Text, अन्य Elements या दोनों हो सकते हैं।
- XML Case Sensitive होता है, इसलिए <Book> और <book> अलग माने जाते हैं।



Example :

```
<bookstore>
  <book>
    <title> XML Guide </title>
    <author> BK LearnX </author>
    <price> 450 </price>
  </book>
</bookstore>
```

Notes :

- Root Element एक ही होता है।
- हर Start Tag का एक End Tag होना चाहिए।
- Proper Nesting अनिवार्य है।

2. XML ATTRIBUTE (विशेषता)

- Attribute किसी Element के बारे में अतिरिक्त जानकारी प्रदान करते हैं।
- Attribute हमेशा Start Tag के अंदर लिखे जाते हैं।
- Attribute का कोई End Tag नहीं होता।
- Attribute Name के बाद = लगाकर Value को Double Quotes (") में लिखा जाता है।

Example :

```
<book id="101" category="programming" available="yes">
  <title> XML Guide </title>
  <author> BK LearnX </author>
  <price> 450 </price>
</book>
```

Example Explanation :

Attribute	Meaning
id = "101"	Book की Unique ID
category = "programming"	Book की Category
available = "yes"	Book उपलब्ध है या नहीं

Key Points :

- ✓ Attribute का उपयोग Additional Information देने के लिए होता है।
- ✓ Attribute Values को " " (Double Quotes) में लिखना Standard है।
- ✓ एक Element में Multiple Attributes हो सकते हैं।

3. XML NAME SPACE (नाम स्थान)

- जब अलग-अलग XML Vocabulary (शब्दावली) के Elements को एक ही Document में Use करना होता है, तो उनकी पहचान के लिए Name Space का उपयोग किया जाता है।
- Name Space नाम के माध्यम से Elements और Attributes के नामों में टकराव (conflict) को रोकता है।
- Name Space का उपयोग xmlns Attribute के द्वारा किया जाता है।

Example : Default Namespace

```
<bookstore xmlns="http://www.bklearnx.com/book">
  <book>
    <title> XML Basics </title>
    <author> BK LearnX </author>
  </book>
</bookstore>
```

Example : Multiple Namespace

```
<bookstore xmlns:bk="http://www.bklearnx.com/book"
  xmlns:au="http://www.bklearnx.com/author">
  <bk:book>
    <bk:title> XML Guide </bk:title>
    <au:authorName> BK LearnX </au:authorName>
  </bk:book>
</bookstore>
```

Key Points :

- ✓ xmlns : Default Namespace define करता है।
- ✓ xmlns:prefix : Prefix के साथ Namespace define करता है।
- ✓ Prefix का उपयोग Elements और Attributes के साथ किया जाता है (जैसे bk:title)।

Benefits of Namespace :

- ✓ Naming Conflicts को रोकता है।
- ✓ Large Applications में अलग-अलग Vocabulary को Manage करना आसान बनाता है।
- ✓ Reusability और Maintainability बढ़ाता है।

Summary :

- ★ XML Elements Document की Structure बनाते हैं।
- ★ XML Attribute Elements की अतिरिक्त जानकारी प्रदान करते हैं।
- ★ XML Namespace अलग-अलग XML Vocabulary के बीच Naming Conflicts को रोकता है और Document को Organized बनाता है।

SYNTAX RULES, XML DTD AND XML SCHEMA

1. XML SYNTAX RULES (सिंटेक्स नियम)

XML Document को सही और Valid बनाने के लिए कुछ नियमों का पालन करना बहुत जरूरी होता है।

- ① XML Document की एक ही Root Element होता है।
- ② हर Start Tag का एक End Tag होना चाहिए।
- ③ XML Tags Case Sensitive होते हैं।
- ④ Tags Properly Nest (सही तरीके से Nested) होने चाहिए।
- ⑤ Attribute Value हमेशा Double Quotes (" ") में लिखे जाते हैं।
- ⑥ XML Document की शुरुआत XML Declaration से हो सकती है।
- ⑦ Comments `<!-- ... -->` के अंदर लिखे जाते हैं।
- ⑧ Special Characters जैसे `<`, `>`, `&`, `'`, `"` को Entity References द्वारा लिखते हैं।
- ⑨ XML File की Extension .xml होनी चाहिए।

Example of Well-Formed XML :

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book id="101">
    <title>XML Basics </title>
    <author>BK LearnX </author>
    <price>450 </price>
  </book>
</bookstore>
```

Incorrect (गलत) Example :

<pre><bookstore> <book> <title>XML </title> </bookstore></pre>	→ Root का End Tag missing → book का End Tag missing → Proper Nesting नहीं है
--	--

2. XML DTD (Document Type Definition)

DTD XML Document की Structure को Define करता है। यह बताता है कि Document में कौन-कौन से Elements होंगे, उनका Order क्या होगा, और कौन से Elements Empty होंगे।

(A) Internal DTD

```
<?xml version="1.0"?>
<!DOCTYPE book [
  <!ELEMENT book (title, author, price)>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
]>
<book>
  <title>XML </title>
  <author>BK LearnX </author>
  <price>450 </price>
</book>
```

(B) External DTD

→ File : book.dtd

```
<!ELEMENT book (title, author, price)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

→ File : book.xml

```
<?xml version="1.0"?>
<!DOCTYPE book SYSTEM "book.dtd">
<book>
  <title>XML </title>
  <author>BK LearnX </author>
  <price>450 </price>
</book>
```

DTD Declarations

- `<!ELEMENT name (content model)>`
→ Element को Define करता है।
- `(#PCDATA)` → Text Data
- `(a, b, c)` → Sequence
- `(a | b)` → Choice
- `(a?)` → Optional
- `(a*)` → Zero or More
- `(a+)` → One or More
- `<!ATTLIST element attribute type default>`
→ Attribute को Define करता है।

3. XML SCHEMA (XSD - XML Schema Definition)

XML Schema DTD का Advanced Version है। यह XML Document की Structure के साथ-साथ Data Type, Default Value, Constraints आदि को Define करता है। यह XML को ज्यादा Powerful और Flexible बनाता है।

Example : book.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="author" type="xs:string"/>
              <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
            <xs:attribute name="id" type="xs:int" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Schema Features

- ✓ Strong Data Type Support (string, int, decimal, date, etc.)
- ✓ Min/Max Value, Length, Pattern आदि को Define कर सकते हैं।
- ✓ Namespace Support करता है।
- ✓ Reusability और Modularity बेहतर होती है।

Example XML with Schema

```
<?xml version="1.0"?>
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="book.xsd">
  <book id="101">
    <title>XML Basics </title>
    <author>BK LearnX </author>
    <price>450.00 </price>
  </book>
  <book id="102">
    <title>XML Advanced </title>
    <author>BK LearnX </author>
    <price>550.50 </price>
  </book>
</bookstore>
```

DTD vs XML Schema

Feature	DTD	XML Schema
Language	SGML based	XML based
Data Type Support	Limited	Strong (string, int, decimal, date, etc.)
Namespaces	Not Supported	Fully Supported
Flexibility	Less Flexible	More Flexible
Error Handling	Less	Better
Use	Simple Documents	Complex & Enterprise Level Applications

Summary

- * XML Syntax Rules को Follow करना जरूरी है वरना Document Well-Formed नहीं बनेगा।
- * DTD Document की Structure को Define करता है।
- * XML Schema More Powerful है और Data Validation बेहतर तरीके से करता है।

RSS FEED, JSON INTRODUCTION AND USES

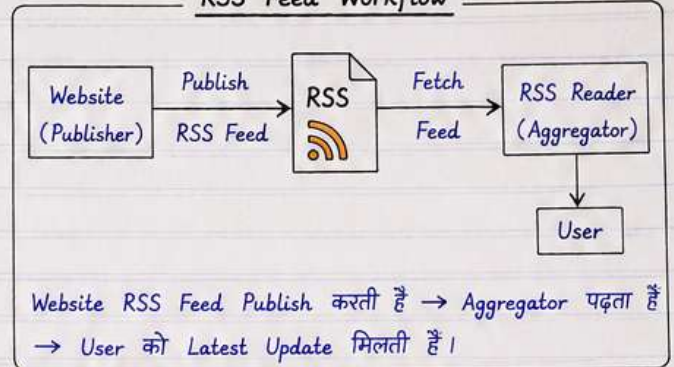
1. RSS FEED (Really Simple Syndication)

- RSS एक XML based format होता है जिसका उपयोग Web Site के Frequently Updated Content को Publish करने के लिए किया जाता है।
- RSS Feed के द्वारा Website अपना Latest Content Subscribers या Aggregator को भेजती है।
- User RSS Reader या Aggregator के माध्यम से सभी पसंदीदा Websites के Updates एक जगह प्राप्त कर सकता है।
- RSS का पूरा नाम Really Simple Syndication है।

RSS Feed Structure (Example)

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0">
  <channel>
    <title> BK LearnX </title>
    <link> https://bklearnx.com </link>
    <description> Education Related Updates </description>
    <item>
      <title> New Course on jQuery </title>
      <link> https://bklearnx.com/jquery </link>
      <description> Learn jQuery Basics </description>
      <pubDate> Sun, 19 May 2024 10:00:00 GMT </pubDate>
    </item>
  </channel>
</rss>
```

RSS Feed Workflow



RSS के मुख्य तत्व

- <rss> : RSS Document का Root Element
- <channel>: Channel Information को Define करता है
- <title> : Channel का Title
- <link> : Channel की Link
- <description> : Channel का Description
- <item> : Individual News/Blog Item
- <pubDate> : Item की Publication Date

Advantages (लाभ)

- ✓ Easy to Publish and Subscribe
- ✓ Website के Traffic को बढ़ाता है
- ✓ Content Distribution आसान होता है
- ✓ Users को Latest Update तुरंत मिलती है

2. JSON INTRODUCTION

- JSON का पूरा नाम JavaScript Object Notation है।
- JSON एक Lightweight Data-Interchange Format है।
- यह Human Readable और Machine Parseable Format होता है।
- JSON Data को Key-Value Pairs के रूप में Store करता है।
- यह Language Independent है और सभी Programming Languages में Support करता है।

JSON की विशेषताएँ

- ✓ Lightweight
- ✓ Easy to Read and Write
- ✓ Fast Data Exchange
- ✓ Language Independent
- ✓ Compatible with JavaScript
- ✓ Supports Arrays and Objects

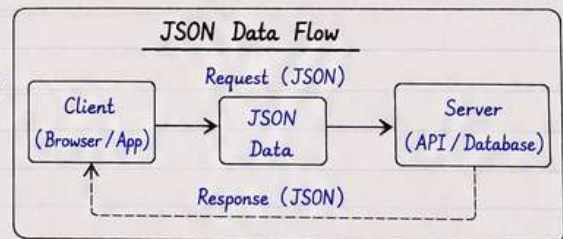
JSON Example

```
{
  "id": 101,
  "name": "Braj Kant",
  "course": "PGDCA",
  "subjects": ["BCIT", "DCCN", "IWT"],
  "active": true,
  "marks": {
    "python": 88,
    "dbms": 91
  }
}
```

3. USES OF JSON (उपयोग)

- Web Applications में Server और Client के बीच Data Exchange करने के लिए।
- RESTful APIs में Data Transfer के लिए (Request और Response में JSON का उपयोग)।
- Web Page में Dynamic Data Display करने के लिए (AJAX के साथ)।
- Configuration Files बनाने के लिए।
- Data Storage और Data Interchange के लिए।
- Mobile Applications में Lightweight Data Format के रूप में।

JSON Data Flow



RSS vs JSON (Comparison)

Feature	RSS FEED	JSON
Full Form	Really Simple Syndication	JavaScript Object Notation
Format	XML Based	Text Based (Key-Value)
Purpose	Content Publish और Subscribe के लिए	Data Exchange के लिए
Structure	Predefined (rss, channel, item)	Flexible (Object, Array)
Readability	कम (XML tags अधिक)	अधिक (Lightweight)
Use	Blog, News, Updates	APIs, Web Apps, Config, Data Transfer
Standard	W3C Standard	ECMA Standard

JSON v/s XML, JSON SYNTAX

1. JSON v/s XML (Comparison)

Feature	JSON	XML
Full Form	JavaScript Object Notation	eXtensible Markup Language
Format	Lightweight, Text Based	Markup Based, Verbose
Syntax	Easy and Simple	Complex and Verbose
Data Type	Supports Objects, Arrays, Number, String, Boolean, Null	Does not support specific data types
Readability	Easy to read and write	Less readable due to tags
File Size	Smaller	Larger
Parsing Speed	Faster	Slower
Browser Support	Native support in JavaScript	Requires XML Parser
Human Readable	Yes	Yes
Use	Web APIs, Data Exchange, Config Files	Configuration, Data Storage, Enterprise Applications
Example	<code>{"name": "Brajkant", "age": 23}</code>	<code><user><name>Brajkant</name><age>23</age></user></code>

2. JSON SYNTAX

- JSON Data को Key-Value pairs के रूप में Store करता है।
- JSON में Data का Format Object {} या Array [] के रूप में होता है।
- Key हमेशा Double Quotes (") में होती हैं।
- String Value भी Double Quotes (") में होती हैं।
- Objects के भीतर Order महत्वपूर्ण नहीं होता है।
- Array में Values क्रमबद्ध (Ordered) होती हैं।
- JSON में Comments नहीं लिखे जा सकते।

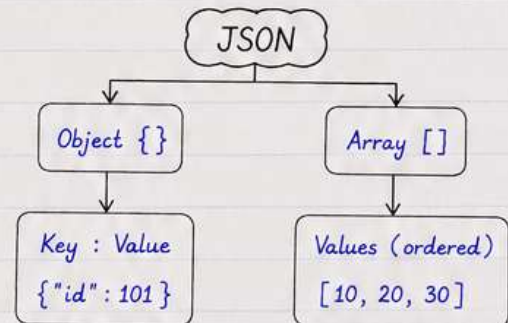
JSON Data Structure

Object : { key : value }

Array : [value, value, ...]

3. JSON SYNTAX RULES

- ① JSON Data {} (Object) या [] (Array) में होता है।
- ② Key हमेशा String (") में होती हैं।
- ③ Key और Value के बीच Colon (:) होता है।
- ④ Key-Value pairs के बीच Comma (,) होता है।
- ⑤ अंतिम Key-Value pair के बाद Comma नहीं लगाते।
- ⑥ Array [] में Values Comma से अलग होती हैं।
- ⑦ Nested Object या Array को Allow करता है।



4. JSON EXAMPLES

(A) Simple JSON Object

```
{
  "id"    : 101,
  "name"  : "Brajkant",
  "course": "PGDCA",
  "active": true
}
```

(B) JSON Array

```
[
  "HTML",
  "CSS",
  "JavaScript",
  "Python"
]
```

(C) Nested JSON

```
{
  "student" : {
    "id"    : 101,
    "name"  : "Brajkant",
    "marks" : [85, 90, 78],
    "active": true
  }
}
```

5. KEY POINTS

- ✓ JSON Lightweight और Fast Data Exchange के लिए बहुत उपयोगी है।
- ✓ Web APIs (RESTful Services) में JSON का सबसे अधिक उपयोग होता है।
- ✓ JSON JavaScript के साथ अच्छी तरह Integrate होता है।
- ✓ Configuration Files, Mobile Apps, Databases में JSON बहुत लोकप्रिय है।

याद रखें :

- * Key हमेशा Double Quotes में।
- * String Value भी Double Quotes में।
- * Comma सही जगह पर लगाएँ।
- * अंतिम Item के बाद Comma नहीं।